

PAL Manual

This manual starts with a small working model and then explains how PAL's pieces fit together. Use the API Guide for compact behavior summaries and the API Reference for signatures.

1. The model loop

A PAL model normally has three parts: **Setup** creates state, **Step** changes state, and **Draw** presents it. PAL does not impose a model class or a scheduler. You choose the update order and when to render.

```
import PythonAutomataLibrary as pal

@pal.njit(cache=True)
def Step(pop: pal.PopGrid):
    # Apply a synchronous increment to every site.
    for i in range(len(pop)):
        pop.Add(1, i)
    pop.Update()

def main():
    pop = pal.NewPopGrid((40, 40))
    for _ in range(10):
        Step(pop)
    print(pop.GetPop())

if __name__ == "__main__":
    main()
```

The example contains 1,600 sites and runs 10 increments per site, so the final total population is 16,000. Add accumulates changes; Update applies them together. That distinction is useful when one site's update must not affect its neighbors during the same timestep. The printed value is 16000; every site contains 10.

PAL uses **safe mode** by default. Develop and validate your model in this mode. If profiling shows that validation overhead matters, call `pal.FastMode()` before creating any PAL objects to remove selected performance-sensitive checks.

2. Choose the right representation

A `Grid` holds one typed value per site. An `AgentGrid` holds individually identifiable agents, optionally with per-agent properties. A `PopGrid` holds integer counts without individual identity. A `PDEgrid` holds a continuous scalar field and transport operations. These can be combined: for example, agents consume a PDE field while population counts accumulate in a separate `PopGrid`.

Use a tuple of dimensions when constructing a spatial grid. Negative dimensions mean periodic wrapping on that axis, so `(-40, 40)` wraps x but not y. `AgentGrid` also supports an empty dimension tuple for a nonspatial population.

Choosing a representation by scale

Individual agents are useful when identity, history, or per-agent properties affect behavior. If agents are interchangeable within a site, a `PopGrid` replaces many handles with one integer count per site and can be much cheaper to update. Its optional `capacity` is a per-site population cap, not a cap on the total population. A `Grid` is appropriate for a site attribute that is not a population, such as a terrain type or a fixed mask. Use `PDEgrid` for a field whose values change continuously through diffusion or advection. A hybrid model can use different representations for different scales, but must explicitly define how state is exchanged between them.

Choose a representation based on what the model must preserve, not just on the number of sites. Converting individual agents to counts loses individual identity and history; representing a count as a continuous field can lose integer and stochastic behavior. Those are modeling assumptions, not implementation details.

3. Coordinates and neighborhoods

A grid has integer lattice sites and may also store agents at continuous positions. A **site index** i is an integer identifying a lattice cell; lattice coordinates such as (x, y) identify that cell on each axis. An agent's continuous coordinates are different: lattice site (x, y) spans continuous positions $[x, x+1) \times [y, y+1)$, and its center is $(x+0.5, y+0.5)$. Thus an agent created or moved with `NewAgentSQ(x, y)` or `MoveSQ(..., x, y)` has integer site coordinates $XSQ=x, YSQ=y$, but continuous coordinates $X=x+0.5, Y=y+0.5$. An agent placed with `NewAgent` or `Move` instead keeps the supplied continuous coordinates while its XSQ/YSQ values identify the containing lattice site. In 2D, `ToI(x, y)` converts lattice coordinates to a linear site index, and `ItoX(i)` and `ItoY(i)` convert that index back to lattice coordinates. Do not confuse the linear site index, lattice coordinates, and continuous agent coordinates. Grid geometry is exposed as attributes such as `xDim, yDim, nDims`, and `wrapX`, not methods.

PAL provides `MooreHood`, `VonNeumannHood`, and `CircleHood`. They return relative integer offsets. `grid.Hood(hood, x, y)` maps those offsets to **lattice coordinates** (a scalar x in 1D, coordinate tuples in 2D/3D), wrapping periodic axes and omitting sites outside nonperiodic axes. It preserves the order and duplicates of the supplied offsets. `grid.Box(...)` instead iterates coordinates in a half-open rectangular region; in 2D it also yields (x, y) tuples, not linear indices. A box wider than a wrapped axis can also visit a site more than once. This distinction matters when using the results for indexing versus coordinate-based operations.

4. Individual agents

An agent is an integer handle owned by an `AgentGrid`, distinct from both its lattice site and its continuous position. Create it with `NewAgentSQ` for a lattice site or `NewAgent` for a continuous position. `NewAgentSQ` and `MoveSQ` accept either a linear site index or lattice coordinates; `NewAgent` and `Move` use continuous coordinates. An agent can move within a site without changing its lattice site. Remove it with `Dispose`.

`agents.All()` returns a snapshot of living handles. That allows creation and disposal during an iteration without changing the iteration's current membership. On a nonstackable grid, safe mode rejects attempts to place two agents at the same site. Use `isStackable=True` only when multiple occupancy is part of the model.

Iterating nearby agents

Use `AgentsAt(x, y)` to visit the agents occupying one lattice site. For a continuous search around a point, `AgentsInRadius(rad, x, y)` yields $(agent, dx, dy, distSq)$ in 2D. The displacement components account for periodic wrapping and $distSq$ is squared distance. In 1D the tuple is $(agent, dx)$; in 3D it is $(agent, dx, dy, dz, distSq)$. Pass `exclude=agent` to omit an agent from its own neighborhood search.

Both iterators work in Python and inside `@pal.njit` loops. Python iteration materializes matching handles first; this does not permit structural modification of the grid during iteration. Safe mode checks for structural changes while iterating, whereas fast mode omits the check. For a loop that creates or disposes agents, iterate a snapshot from `agents.All()` instead.

For example, this Python-side search finds the agent across a periodic x boundary:

```
import PythonAutomataLibrary as pal

agents = pal.NewAgentGrid((-10, 10))
agent = agents.NewAgent(9.0, 2.0)
nearby = list(agents.AgentsInRadius(1.5, 0.0, 2.0))
assert len(nearby) == 1
found, dx, dy, distSq = nearby[0]
assert found == agent
assert dx == -1.0 and dy == 0.0 and distSq == 1.0
assert list(agents.AgentsInRadius(1.5, 0.0, 2.0, exclude=agent)) == []
```

Agent birth/death timestep

`Examples/Agents/SaveLoadModel.py` is a complete agent-based model using a nonstackable 2D `AgentGrid`. Its compiled `Step` iterates `grid.All(shuffle=True)`, removes an agent on a death draw, and otherwise attempts reproduction on a second draw. To reproduce, it scans the Moore neighborhood, stores empty site indices in a reusable `IList`, and creates an agent at a randomly selected empty site.

The distinction between **snapshot iteration** and **current occupancy** matters here. `All()` fixes which agents get a turn at the beginning of the step; newborn agents do not act until the next step. But neighborhood occupancy is

queried when each parent acts, so births and deaths earlier in the same step can change later agents' options. Shuffling changes that order and therefore can change outcomes even with identical initial occupancy. This is a sequential stochastic agent update, unlike the simultaneous queued `PopGrid.Add/Update` pattern below.

The example's `Setup`, `Step`, and `Draw` functions are `@pal.njit(cache=True)` functions. The Python main loop owns the window, timestep, and checkpoints; compiled functions handle repeated model operations. Use the source example for the complete runnable program rather than duplicating its graphics and checkpoint lifecycle here.

5. Simultaneous population and field updates

Both `PopGrid` and `PDEgrid` distinguish current state from pending changes. Direct indexing changes current state immediately; `Add` queues a delta, and `Update` applies queued changes. `Reset` clears both current state and pending changes.

A `PDEgrid` also provides diffusion and advection. Set the timestep and spatial spacings with `SetTimeSpaceStep(dt, dx, dy, dz)` before transport operations. Cartesian, interface-based, masked, ADI, and radial diffusion variants are available; choose the method that matches the model's geometry and numerical assumptions. Cartesian boundaries may be uniform or vary across a face; wrapped axes are periodic instead. Validate timestep stability and boundary behavior in safe mode before optimizing.

Advection versus diffusion example

`Examples/Diffusibles/ReactionDiffusion2D.py` compares two `PDEgrid` fields on the same 20 x 10 lattice. Each timestep sets a short vertical source at `x=10` in both fields. One field then calls `Advection(vx=0.01, vy=0.01, xMaxBC=0, yMaxBC=0)`; the other calls `DiffusionADI(0.01, xMaxBC=0, xMinBC=0)`. Each transport call is followed by its own `Update()`.

This makes the physical distinction visible: advection transports the field along a velocity, while diffusion spreads it down concentration gradients. Boundary concentrations also play different roles: diffusion couples them to the boundary gradient, while advection uses them only where velocity points into the domain; an advective outflow uses the interior upwind value. The explicit boundary arguments are part of the model, not cosmetic rendering settings. `Draw` maps each field's current values through `ColorScale` into separate pixel windows. As in the agent example, `Step` and `Draw` are compiled, while `main` manages windows and iteration.

The example is a qualitative comparison, **not** a convergence or mass-conservation test: it continually resets source cells and uses different boundary conditions for the two fields. For quantitative validation, remove the source, use equivalent boundary conditions, and check expected mass balance and grid/timestep convergence.

A hand-checkable field gradient

`Examples/Diffusibles/Gradient2D.py` provides a simpler numerical check than a transport simulation. It assigns `grid[x, y] = y / grid.yDim` on a 10 x 10 `PDEgrid`. The centered finite differences at (5, 5) are therefore

```
import PythonAutomataLibrary as pal

field = pal.NewPDEgrid((10, 10))
for x in range(10):
    for y in range(10):
        field[x, y] = y / 10

dx = (field[6, 5] - field[4, 5]) / 2
dy = (field[5, 6] - field[5, 4]) / 2
assert abs(dx) < 1e-6
assert abs(dy - 0.1) < 1e-6
```

The field is constant in `x` and linear in `y`, so its expected discrete gradients are 0 and 0.1. The assertions allow floating-point storage and arithmetic error. This verifies indexing and a finite-difference calculation without introducing boundary conditions or timestep integration. `Examples/Diffusibles/Gradient2D.py` performs the same calculation inside `@pal.njit`.

Numerical update order

The timestep order is part of the model. For example, queue all births and deaths in a `PopGrid` before calling `update()` if changes must be simultaneous. Reading current counts while queuing deltas then uses the same starting population throughout the step. By contrast, assigning directly to `pop[x, y]` changes the value that subsequent calculations read. Mixing immediate assignments and queued changes is possible, but their order then changes the model.

For a `PDEgrid`, transport routines accumulate changes that become current at `update()`. Check timestep stability and convergence by varying the timestep and grid spacing. Safe-mode checks do not replace numerical validation.

Consider two sites with populations 10 and 0. If a transfer moves 5 from site 0 to site 1, queue -5 and +5 before a single `update()`; the result is (5, 5) and total population remains 10. Applying one change immediately would alter what later calculations read and can make an intended simultaneous update order-dependent.

For a small hand-calculated timestep, check the properties the model requires: counts stay nonnegative, internal transfers conserve total population, totals change only through explicit births, deaths, or boundary flux, and results are independent of site visitation order when the update is meant to be simultaneous.

6. Compiled model code

Decorate substantive computational functions with `@pal.njit(cache=True)` so compiled code can be reused across runs. PAL expands some model constructs before passing the function to Numba, including neighborhood iteration and source-aware safe-mode diagnostics. Annotate PAL arguments with their public types, such as `pal.AgentGrid` or `pal.PDEgrid`, for clarity and compiler support.

Keep model state in PAL objects and use `pal.Seed`, `pal.Random`, `pal.RandInt`, and `NewMultinomial` for reproducible PAL random sampling. Run a model in safe mode first. Fast mode removes selected checks; it does not make invalid indices, dead handles, or unstable numerical steps valid.

Testing a timestep

Test a model's update function on a grid small enough to inspect by hand, including an edge or wrapped boundary rather than only interior sites. Use invariants appropriate to the object: closed diffusion conserves mass within floating-point tolerance; agent moves preserve valid live handles and occupancy rules; population updates satisfy the conservation and nonnegativity checks from section 5.

When a model uses both Python orchestration and compiled `@pal.njit` updates, test both paths; passing in Python does not establish compiled behavior. Include safe and fast runs in regression testing, but use safe-mode exceptions to diagnose invalid operations rather than expecting fast mode to detect them.

Random streams and repeatability

`pal.Seed(seed)` initializes PAL's random stream. `pal.Random()` draws a uniform value and `pal.RandInt(max)` draws an integer from `[0, max)`. PAL random draws from Python and compiled PAL code consume the same stream, so **the order of calls matters**. Repeating a run requires the same seed, initial state, model parameters, and random-call order; merely reseeding halfway through a simulation does not reconstruct the prior model state.

For comparisons between two algorithms, seed each run separately and record the seed. If one implementation makes additional random draws, subsequent results can diverge even when both start from the same seed. External random libraries (such as NumPy's generators) have independent state and must be seeded or checkpointed separately.

7. Drawing and output

`StartPixWindow(xDim, yDim, scale=1, title='PAL', headless=False)` returns `(pix, window)`. Write packed RGB colors such as `0xFF0000` to `pix[x, y]`, then call `window.Update()` to publish the frame. `window.Save(path, block=True)` waits for the image to be written; use `window.Close()` to release resources. The pixel buffer also supports linear indices, slices, and writes from compiled code. For off-screen images, use `headless=True` (requires the relevant output dependencies). GIF recording uses `window.StartGif(path, delay=100)`, `window.AddGifFrame(block=False)`, and `window.StopGif()`; update the window before capturing each frame.

`StartOpenGLWindow` provides geometric 2D/3D drawing rather than a writable pixel lattice. Use its drawing object for shapes and scene controls; its window uses the same update, image/GIF output, and close lifecycle described above. `headless=True` is available for off-screen rendering but still requires a working OpenGL backend. See the [API Guide](#) or [Reference](#) for the drawing-method inventory.

Rendering and simulation are separate. A model can run without a window, draw every timestep, or draw only selected checkpoints.

8. Checkpointing and reproducibility

PAL's native-backed model objects support Python pickling, so a long-running model can save its state and resume without retaining native pointers from the original process. A checkpoint must include **all** state needed for continuation: every grid, relevant parameters, the current timestep, and any additional state managed by the model. Saving only one grid does not preserve a coupled simulation. Keep the code version and parameter configuration alongside the checkpoint so that results can be interpreted later.

A PAL object checkpoint does **not** serialize PAL's random stream. If exact stochastic continuation matters, the model must reconstruct the intended random state separately; external RNGs such as NumPy's are independent as well. Verify checkpoint behavior with a short uninterrupted run and a checkpoint-and-resume run rather than assuming object state alone reproduces the same future random draws.

A minimal checkpoint can be saved and resumed without opening a window:

```
import pickle
import PythonAutomataLibrary as pal

pop = pal.NewPopGrid((5,))
pop[1] = 10
pop.Add(7, 1) # Pending change is part of the checkpoint.
state = {"pop": pop, "step": 3}
checkpoint = pickle.dumps(state)

restored = pickle.loads(checkpoint)
restored["pop"].Update()
assert restored["step"] == 3
assert restored["pop"][1] == 17
```

`Examples/Agents/SaveLoadModel.py` serializes a dictionary containing an `AgentGrid`, an `IList`, and the timestep, then restores all three before continuing. PAL object checkpoints do not include unrelated Python state or external RNG streams.

9. How PAL works

PAL separates its public Python interface from a native C core. Python constructors create native-backed model state, and thin Numba-compatible wrappers let compiled model code operate on that state without repeatedly returning to Python. Before Numba compilation, `pal.njit` transforms PAL-specific constructs such as `Hood` and `Box` iteration and adds source-aware diagnostics where annotations provide enough type information. The public Protocol classes describe the annotation/autocomplete surface while safe and fast concrete implementations provide runtime behavior.

This design favors fast repeated timesteps, but the first compiled call may include substantial Numba compilation time. Measure **cold compilation** separately from **steady-state execution**. Safe mode includes additional validation and source-aware diagnostics; fast mode removes some of that overhead. A change that improves one benchmark may regress another, so measure both modes on representative workloads.

10. Testing PAL

PAL's `Testing/` suite exercises public behavior across geometry and wrapping, `Grid/IList/Multinomial/RNG` operations, `AgentGrid` and `PopGrid` state changes, PDE transport and boundaries, compiled Python/Numba parity, visualization, integrated models, and historical regressions. Many tests use independent invariants or small reference calculations rather than reproducing PAL's implementation.

Run the full suite from the repository root in separate safe and fast processes:

```
PAL_TEST_MODE=safe python -m pytest -q Testing
PAL_TEST_MODE=fast python -m pytest -q Testing
```

PAL locks its process-wide mode at first construction, so safe and fast should not be combined in one Python process. Fast mode intentionally skips some validation tests. The CI workflow runs both modes on Linux, verifies that the generated API Reference is current, runs the API benchmark after successful tests, and separately runs representative examples in safe and fast modes on Windows. Headless OpenGL tests require the relevant graphics backend.

When adding or changing model code, start with the smallest test that expresses the intended public behavior, then run the affected test file before the full suite. Prefer conservation laws, coordinate identities, exact small examples, and Python/compiled agreement over tests that duplicate PAL's internal formulas.

11. Where to go next

Use the Cheatsheet for short code patterns, the API Guide for behavioral contracts, and the generated API Reference for current signatures. `Examples/` contains complete models; the Testing guide describes regression coverage and how to run the suite.